Capteurs de vision: Fusion multi-capteurs

Hind Laghmara et Robin Condat

1 Introduction

Aujourd'hui, place à l'acquisition multi-capteurs. Depuis le début, à chaque TP, on utilise uniquement un seul capteur. C'est cool, mais on ne va pas très loin non plus. Cependant, faire une acquisition avec plusieurs capteurs, ce n'est pas aussi simple. L'objectif principal est donc tout trouvé : coder votre première acquisition impliquant 2 capteurs, à savoir ici la caméra et le LiDAR.

Objectifs

- Configurer et interfacer une caméra, un LiDAR sur Raspberry Pi.
- Acquérir et enregistrer les données des différents capteurs.
- Synchroniser les flux de données multi-capteurs.

Matériel requis

- Raspberry Pi 4 Model B
- Caméra Raspberry Pi Camera Module 3 / Rev3
- RPLidar A1M8
- Un bouton
- Une LED

2 Mise en place

Pour ce TP, on reprend le même setup que le TP précédent, à savoir votre Raspberry branchée à une caméra, une LED et à un bouton poussoir. On rajoute à cela le LiDAR, connecté en USB.

Au lieu de repartir de la précédente archive du TP5, on va repartir de zéro avec un nouveau code. Des changements ont été réalisé, notamment sur la gestion du LiDAR (normalement, il n'y a plus le bug relou du TP3), et vous n'avez plus besoin d'utiliser le package non maintenu depuis 2016 `rplidar-roboticia`.

- Branchez le bouton et la LED à la Raspberry.
- Téléchargez le code disponible sur Moodle, décompressez l'archive zip et renseignez le chemin de votre dossier dans la variable d'environnement `PYTHONPATH` :

```
export PYTHONPATH=/chemin/vers/votre/dossier/TP:$PYTHONPATH
```

- Parcourez l'archive de code avec les **nombreux** fichiers, et recensez de votre côte les différentes fonctions disponibles (notamment dans les dossiers `core/`, `sensors/` et `utils/`), la nouvelle classe LiDAR (`sensors/LIDAR/lidar.py`) ainsi que les paramètres de configuration (dans `config/`)
- Complétez les fonctions suivantes, avec le code réalisé durant les derniers TPs:
  - `filter_{angle|range|quality}` dans `sensors/LIDAR/lidar.py`
  - `polar_to_cartesian` dans `sensors/LIDAR/lidar.py`
  - `measure_distance` dans `scripts/measure_distance.py`
  - Les constantes et paramètres dans `config/`
- Vérifiez votre bonne implémentation grâce aux scripts mis à jour `scripts/single_scan.py`, `scripts/stream_scan.py` et `scripts/measure_distance.py`

⇒ *L'affichage du nuage de points en direct a beaucoup changé. Il est certes moins joli, mais est beaucoup plus léger, ce qui permet à la Raspberry de respirer...*

3 Acquisition LiDAR

Nous avions à l'issue des dernières séances un super code pour l'acquisition caméra : `scripts/capture_video.py` (le code de l'archive est corrigé, rien n'est à compléter). Il nous faut maintenant un super code pour l'acquisition LiDAR !

- Complétez le fichier `scripts/capture_lidar.py`, afin de réaliser un code similaire à l'acquisition caméra.

  *Note : L'enregistrement des scans à changé par rapport au TP3, on enregistrera chaque scan dans un fichier CSV. Une fonction **BIEN UTILE** est disponible dans la classe LIDAR*

- Testez votre script complété en réalisant une acquisition LiDAR

4 Acquisition multi-capteurs

Vous avez vos deux scripts pour réaliser une acquisition multi-capteurs (caméra et LiDAR).

- Exécutez dans 2 terminaux séparés vos deux scripts d'acquisition.

⇒ Pourquoi l'acquisition ne fonctionne pas ici ?

En effet, ce n'est pas parce que vos deux scripts sont fonctionnels qu'ils peuvent être utilisés en même temps. On pourrait ré-écrire les scripts pour utiliser deux boutons et LEDs différentes, mais ça complexifie la tâche. On pourrait également récupérer un des deux codes, et rajouter l'acquisition du 2ème capteur au moment de l'acquisition du 1er. Cependant, nous avons des capteurs qui fonctionnent à des fréquences différentes:

Capteur	Fréquence
Caméra	$\approx$ 30 Hz
LiDAR	$\approx$ 5–10 Hz

Hors, en forçant la récupération des données en même temps pour les deux capteurs, vous risquez d'avoir au choix des données dupliquées fausses, ou des données manquantes. Bref, c'est pas fou fou...

La solution ici va être d'acquérir les données des différents capteurs via **l'utilisation de threads**. Un thread permet d'exécuter plusieurs tâches en parallèle dans un même programme, en partageant les mêmes ressources. Un programme peut lancer plusieurs threads qui s'exécutent en parallèle (ou quasi-parallèle).

Le script `scripts/example_multithreading.py`, que je vous invite à exécuter, réalise un exemple de multithreading avec 2 capteurs fictifs. Vous avez ici deux threads appelant chacun un worker pour le capteur A et le capteur B. Ces deux capteurs ont une fréquence d'acquisition différente:

```
def sensor_a_worker(state, lock, stop_event):
    """
    Simule un capteur A
    """
    ...

def sensor_b_worker(state, lock, stop_event):
    """
    Simule un capteur B
    """
    ...
```

Ces deux workers communiquent avec un état commun `state` partagé:

```
state = {
    "sensor_a": ...,
    "sensor_b": ...
}
```

De ce fait, lorsque les workers veulent communiquer avec le programme principal, ou inversement, l'objet `state` est utilisé. Cependant, afin d'éviter les conflits d'écriture et les incohérences, on utilise un verrou (`lock`):

```
lock = threading.Lock()

with lock:
    state["..."] = ...
```

Et pour arrêter les deux workers, un événement d'arrêt (`stop_event`) est nécessaire:

```
stop_event = threading.Event()
```

Dans notre contexte d'acquisition multi-capteurs, nous allons utiliser deux threads :

- Un thread pour l'acquisition caméra
- Un thread pour l'acquisition LiDAR

Le code principal construira et appellera les deux threads, tout en gérant les débuts et fin d'enregistrements via le bouton, ainsi que l'arrêt total du script. Vous avez à votre disposition le script à compléter `scripts/capture_multisensor.py`.

- Complétez le script avec tout ce qui vous est demandé (il y a beaucoup de lignes à compléter)
- Dès que tout est prêt, ok, et fonctionnel, sortez de la salle pour faire une grande acquisition dans le bâtiment Magellan, avec la caméra et le LiDAR fonctionnant en parallèle.
- Regardez en détail les acquisitions obtenues dans le dossier créé (les fichiers et dossiers créés, le nombre d'images et scans obtenus)

⇒ Combien d'images/scans avez vous obtenu ?

⇒ A partir des fichiers timestamps CSV, quelles sont les fréquences d'acquisitions des deux capteurs ? Comment varient ces fréquences au cours du temps ? (et oui, on fait des stats aussi en Capteurs)

- Exécutez le script `postprocessing/replay_multisensor.py` afin de visualiser votre longue épopée dans l'INSA.

5 Synchronisation

Votre acquisition précédemment réalisée est asynchrone: chaque capteur interagit à sa propre vitesse, indépendamment des autres. Pour de l'acquisition brute, c'est suffisant, mais s'il faut exploiter les données entre elles, ça ne le fait pas du tout.

Tout dépend de la tâche...

Une étape nécessaire est donc la synchronisation des données. Elle se fait souvent *online*, donc pour notre cas, durant l'acquisition. Maintenant, c'est complexe, ça redemande de modifier le code déjà existant, donc on va partir aujourd'hui sur une synchronisation *offline*.

Le principe ici est le suivant :

- On lit les données d'une acquisition précédemment réalisée, et plus particulièrement les fichiers CSV de timestamps
- On définit un capteur de référence (de préférence celui avec la fréquence d'acquisition la plus faible)
- Pour chaque donnée du capteur de référence, on trouve celle de l'autre capteur la plus proche dans le temps (à partir des timestamps)
- Si le temps entre les timestamps de la paire de data est en dessous d'un seuil maximum prédéfini, et que les deux données n'ont pas été déjà utilisés pour former une autre paire, on sauvegarde les datas appairées dans un nouveau dossier d'acquisition synchronisé

A vous de jouer ! Complétez le fichier `postprocessing/synchronization.py` afin de réaliser la synchronisation précédemment décrite.

6 Jamais deux sans trois...

Si vous en êtes là, bravo, car il y avait du travail ! La prochaine étape ici vous permet de prendre de l'avance sur le projet intégratif que vous aurez en fin de semestre. Vous avez pour l'instant plusieurs scripts faisant de l'acquisition simultanée d'une caméra et d'un LiDAR, de la synchronisation offline, et une visualisation "replay" de l'acquisition. A vous de modifier les différents scripts pour ajouter l'utilisation du GPS ou de l'IMU (ou des deux).

Si vous avez un code réalisant l'acquisition brute des 4 capteurs, vous allez avoir une superbe base pour le PI :D